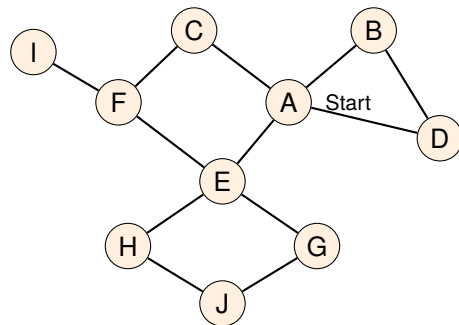


3.3 Breitensuche

Aufgabe 1) Silvesterfeuerwerk

Die Stadt hat Feuerwerksbatterien mit Zündschnüren verbunden. Batterie A wird um 0 s gezündet, jede Zündschnur brennt genau 1 s. Welche Batterien zünden wann?

Zeit	Batterien
0 s	A
1 s	B, C, D, E
2 s	F, G, H
3 s	I, J (zuletzt)



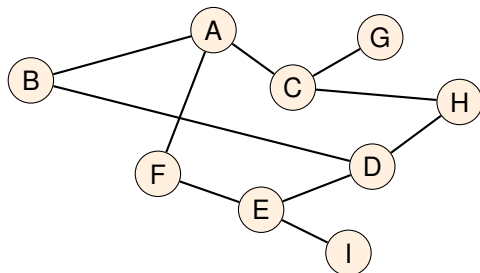
Breitensuche

Die Breitensuche durchläuft (*traversiert*) einen Graphen schichtweise: zuerst alle Nachbarn des Startknotens, dann deren noch nicht besuchte Nachbarn usw. Mit jeder Schicht wird die Pfadlänge um eins größer. Deshalb findet sie in ungewichteten Graphen den Pfad mit den wenigsten Kanten.

Die noch zu besuchenden Knoten stehen in einer Warteschlange: Wer zuerst hineinkommt, kommt zuerst wieder heraus (engl. *first in, first out*). Besuchte Knoten werden markiert, damit kein Knoten doppelt besucht wird – wie die Brotkrumen im Spiel.

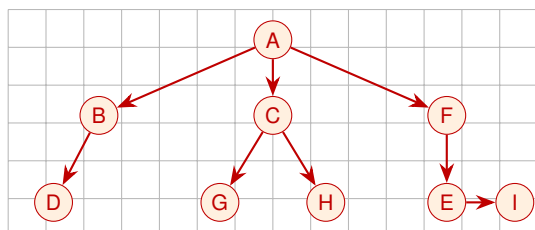
Aufgabe 2)

a) Führe die Breitensuche ab Knoten A durch. Nimm Nachbarn immer alphabetisch. Notiere nach jedem Schritt den aktuellen Knoten und die Warteschlange.



Schritt	aktueller Knoten	Warteschlange danach
0	–	A
1	A	B C F
2	B	C F D
3	C	F D G H
4	F	D G H E
5	D	G H E
6	G	H E
7	H	E
8	E	I
9	I	(leer)

b) Zeichne den *Besuchsbaum*: einen Pfeil von jedem Knoten zu den Knoten, die er in die Warteschlange gestellt hat.



c) Reihenfolge ab Knoten E:

E, D, F, I, B, H, A, C, G

d) Anton behauptet: „Die Breitensuche besucht immer alle Knoten.“ Nimm Stellung.

Falsch: Nur Knoten, die vom Start aus erreichbar sind.

Bei nicht zusammenhängenden oder nur schwach zusammenhängenden Graphen fehlen Knoten.

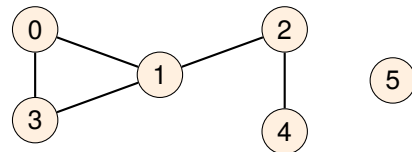
Aufgabe 3) Projekt Breitensuche

a) Ordne jeder Zeile des Pseudocodes die Nummer des passenden Codefragments zu (Website bzw. Datei Codefragmente.java). **b)** Setze die Methode `breitensuche` im Projekt um (TODO 1 bis 3) und teste sie.

Pseudocode <code>breitensuche(start, ziel)</code>	Fragment
aktuellerKnoten $\leftarrow$ start	2
lege aktuellerKnoten in die Warteschlange	1
gib „Reihenfolge: “ aus	7
solange aktuellerKnoten $\neq$ ziel und Warteschlange nicht leer	6
aktuellerKnoten $\leftarrow$ vordersten Knoten aus der Warteschlange nehmen	4
aktuellerKnoten als besucht markieren	8
gib aktuellerKnoten aus	3
für alle Knoten i	5
falls Kante von aktuellerKnoten zu i und i nicht besucht und nicht in der Warteschlange: stelle i hinten an die Warteschlange	9
falls aktuellerKnoten = ziel: „gefunden“, sonst : „nicht gefunden“	10

Aufgabe 4) Projekt Breitensuche: Testen

Im Hauptprogramm entsteht dieser Graph.
Knoten 5 wird mit `knotenHinzufuegen()` ergänzt,
hat aber keine Kante.



a) Sage die Ausgabe voraus, dann teste.

Aufruf	Reihenfolge	gefunden?
<code>breitensuche(3, 2)</code>	3 0 1 2	ja
<code>breitensuche(4, 0)</code>	4 2 1 0	ja
<code>breitensuche(0, 5)</code>	0 1 3 2 4	nein

b) Stolperstein: Verschiebe Fragment 4 (`poll`) an das Ende der Schleife. Was gibt `breitensuche(3, 2)` jetzt aus? Erkläre, was schiefgeht.

„Reihenfolge: 3 3 0“, nicht gefunden. Der Startknoten liegt noch in der Warteschlange und wird ein zweites Mal geholt. Nach dem Holen von Knoten 1 ist die Warteschlange leer, die Schleife endet, bevor Knoten 1 untersucht wird: Knoten 2 wird nie erreicht.